

UADC Script Processing Guide

This document explains the current UADC PoC script processing model. It describes how the scripts interpret binding tables, XPath, Semantic Path, repeated classes, and dimension columns such as **dInvoice** and **dInvoiceLine** to build the target data structures.

Processing Model

The UADC PoC separates processing into two stages.

```
Source syntax XML
-> syntax binding
-> UADC Structured CSV

UADC Structured CSV
-> target syntax binding or semantic binding
-> XBRL GL, PSV, or CSV target view
```

Phase 1 creates the common hierarchical Structured CSV. Phase 2 consumes that Structured CSV and generates target views.

The core idea is that a binding row connects three things:

- a semantic location, written as **semantic_path**;
- a syntax location, written as **xpath** when XML or XBRL XML is involved;
- a Structured CSV column, written as **structured_csv_column** or derived from **semantic_path**.

Class rows use **type=C**. Fact rows use **type=A**. Class rows define the tree and repeated row scopes. Fact rows define values.

Semantic Path And XPath

semantic_path identifies the position of a term in the common UADC semantic model.

```
$.invoice.invoiceNumber
$.invoice.invoiceLine.invoiceLineIdentifier
$.invoice.invoiceLine.itemInformation.itemAttributes.itemAttributeValue
```

The path before the final segment identifies the semantic parent class. The final segment normally becomes the Structured CSV value column in UpperCamelCase.

```
$.invoice.invoiceNumber -> InvoiceNumber
```

```
$.invoice.invoiceLine.invoiceLineIdentifier -> InvoiceLineIdentifier
```

xpath identifies the concrete XML or XBRL XML element or attribute.

```
/Invoice/cbc:ID
/Invoice/cac:InvoiceLine/cbc:ID
/Invoice/cac:InvoiceLine/cbc:InvoicedQuantity/@unitCode
```

When **xpath** ends with an attribute, **element** remains the owner element.

```
element: cbc:InvoicedQuantity
xpath: /Invoice/cac:InvoiceLine/cbc:InvoicedQuantity/@unitCode
```

Predicates in **xpath** select a branch. For example, UBL allowances and charges are distinguished by **cbc:ChargeIndicator=false()** or **cbc:ChargeIndicator=true()**. VAT totals can be selected by a currency predicate.

Structured CSV Dimensions

Structured CSV represents a hierarchy as sparse rows. Dimension columns identify the semantic context of each row.

Typical dimension columns:

```
dInvoice
dVatBreakdown
dInvoiceLine
dInvoiceLineAllowances
dInvoiceLineCharges
dItemAttributes
```

The dimension name is derived from the repeated class column.

```
InvoiceLine -> dInvoiceLine
VatBreakdown -> dVatBreakdown
ItemAttributes -> dItemAttributes
```

A class creates an independent dimension only when its **multiplicity** upper bound is repeated, such as **0..**, **1..**, **n**, **unbounded**, or a number greater than 1.

Examples:

Invoice	1..1	-> dInvoice root only
InvoiceLine	1..*	-> dInvoiceLine
ItemInformation	1..1	-> no independent repeated row scope
ItemAttributes	0..*	-> dItemAttributes

Non-repeated parent classes are still important in **semantic_path** and XPath, but they do not create a new row scope by themselves.

Phase 1 Forward Conversion: UBL XML To Structured CSV

Script:

```
src/syntax_binding.py
```

Main function:

```
write_hierarchical_csv(...)
```

Input:

- UBL Invoice XML.
- Syntax binding CSV, currently
specs/bindings/syntax/EN16931_UBL_Invoice_Syntax_Binding.csv.

Output:

- Phase 1 Structured CSV.
- xBRL-CSV metadata JSON.

Binding Table Interpretation

The syntax binding table contains both class rows and fact rows.

Class row:

```
type=C
semantic_path=$.invoice.invoiceLine
structured_csv_column=InvoiceLine
multiplicity=1..*
xpath=/Invoice/cac:InvoiceLine
```

Fact row:

```

type=A
semantic_path=$.invoice.invoiceLine.invoiceLineIdentifier
structured_csv_column=InvoiceLineIdentifier
xpath=/Invoice/cac:InvoiceLine/cbc:ID

```

The class row tells the converter where the XML context is and whether it repeats. The fact row tells the converter which value to extract inside that context.

Internal Data Structures

read_binding_rows(binding_csv, encoding) returns the raw binding table as:

```
list[dict[str, str]]
```

Each dictionary is one CSV row keyed by the header.

read_binding_layout(...) and **build_layout_from_rows(...)** build **BindingLayout**, the runtime Structured CSV layout derived from the binding table.

Important fields in **BindingLayout**:

- **fieldnames:** **list[str]**: final Structured CSV column order.
- **dimensions:** **list[str]**: dimension columns.
- **field_dimension:** **dict[str, str]**: value column to dimension column.
- **field_by_semantic_path:** **dict[str, str]**: semantic path to Structured CSV value column.
- **semantic_path_dimension:** **dict[str, str]**: semantic class path to dimension column.
- **dimension_xpath:** **dict[str, str]**: dimension column to XML context XPath.
- **dimension_ancestors:** **dict[str, list[str]]**: repeated ancestor dimensions.
- **dimension_repeats:** **dict[str, bool]**: repeat status for each emitted dimension.
- **syntax_sequence_by_field** and **syntax_sequence_by_dimension**: reverse XML ordering hints.

read_bindings(...) builds:

```
bindings: list[Binding]
```

Binding is a dataclass normalized from **type=A** rows. It stores:

- **order**;
- **semantic_path**;
- **xpath**;
- **field**, the Structured CSV value column;
- **dimension**, the row-scope dimension column;

- optional filter fields and values.

Example:

```
Binding(
  semantic_path="$.invoice.invoiceLine.invoiceLineIdentifier",
  xpath="/Invoice/cac:InvoiceLine/cbc:ID",
  field="InvoiceLineIdentifier",
  dimension="dInvoiceLine"
)
```

build_binding_class_tree(rows) builds a tree of **BindingClass** nodes from **type=C** rows.

Each **BindingClass** stores:

- **semantic_path**;
- **xpath**;
- **column**, the class column;
- **dimension**, such as **dInvoiceLine**;
- **repeats**, based on **multiplicity**;
- **children**: **list[BindingClass]**.

The class tree is the semantic and XML context tree used by the converter. It is built from the binding table, not inferred from the source XML.

direct_class_fields(class_path, bindings) groups fact bindings by their direct semantic parent. The result is used as:

```
direct_fields_by_class: dict[str, list[Binding]]
```

This prevents a parent class from extracting descendant facts too early.

Dimension Ownership Resolution

The forward and reverse paths use the same ownership metadata derived from the syntax binding table.

1. **multiplicity_repeats(multiplicity)** recognizes *****, **n**, **unbounded**, or a numeric upper bound greater than 1 as repeating.
2. **build_layout_from_rows(rows)** reads every **type=C** class row. The invoice root becomes **dInvoice**. Other class rows become **dXxx** only when their multiplicity repeats.
3. While reading each **type=A** row, the nested **nearest_dimension(...)** helper walks upward through its **semantic_path** until it finds the nearest class that owns a dimension.
4. The result is stored in **BindingLayout.field_dimension** and copied into each normalized **Binding.dimension** by **read_bindings(...)**.

Therefore a non-repeating child class belongs to its nearest repeating ancestor row. For example, non-repeating item information below a repeating invoice line belongs to **dInvoiceLine**. A repeating child class owns its own dimension and is not merged into its parent row.

The ownership rule is independent of the target XML QName. It is determined by **type=C**, **multiplicity**, and **semantic_path**, while **structured_csv_column** supplies the actual fact and class column names.

XPath Context Recursion

The forward conversion uses XML parent-context recursion. The central nested function inside **write_hierarchical_csv(...)** is:

```
process_class(context, class_node, dimension_values, current_row)
```

Arguments:

- **context: ET.Element**: XML element for the current class.
- **class_node: BindingClass**: semantic class being processed.
- **dimension_values: dict[str, str]**: active dimension values inherited from parent contexts.
- **current_row: dict[str, str] | None**: current Structured CSV row.

At the root, the converter calls:

```
process_class(root_context, class_root, {"dInvoice": "1"}, None)
```

This means the first invoice has **dInvoice=1**.

How A Rows Are Filled

For each class context, **fill_direct_fields(row, context, class_node)** processes only the direct **A** rows for that class.

For each **Binding**:

1. **relative_xpath(binding.xpath, class_node.xpath)** converts the absolute binding XPath into a path relative to the current class context. If the binding points outside that context, the path remains absolute and forward extraction evaluates it from the document root.
2. **get_value(context, relative_xpath, namespaces, root)** extracts text or attribute value.
3. The value is written to **row[binding.field]**.

Example:

Class:

```
$.invoice.invoiceLine  
/Invoice/cac:InvoiceLine
```

Fact:

```
$.invoice.invoiceLine.invoiceLineIdentifier  
/Invoice/cac:InvoiceLine/cbc:ID
```

Relative XPath:

```
cbc:ID
```

The converter extracts **cbc:ID** from the current **cac:InvoiceLine** context and writes it to **InvoiceLineIdentifier**.

How Repeated Dimensions Are Decided

Repeated dimensions are decided from **BindingClass.repeats**, which is derived from the class row **multiplicity**.

When **process_class(...)** sees a child class:

- if the child is non-repeated, it processes that child using the same **current_row**;
- if the child is repeated and has a dimension, it delays that child into **repeated_children**.

After the current row is filled and written if needed, repeated children are processed one occurrence at a time.

For each repeated child occurrence:

1. **class_contexts(parent_context, parent_class, child_class)** finds child XML contexts under the current parent XML context.
2. The occurrence number starts at 1.
3. A new **child_dimensions** dictionary is created by copying parent dimensions.
4. The child dimension is set.
5. A new row is created.
6. The function recurses into the child XML context.

Example:

```
Parent dimensions:  
{  
  "dInvoice": "1"
```

```

}

Second invoice line:
{
  "dInvoice": "1",
  "dInvoiceLine": "2"
}

First item attribute under that line:
{
  "dInvoice": "1",
  "dInvoiceLine": "2",
  "dItemAttributes": "1"
}

```

This is how **dInvoice**, **dInvoiceLine**, and deeper dimension values are determined. The value is the 1-based occurrence number inside the current XML parent context.

Row Creation And Sparse Rows

Rows are dictionaries:

```
dict[str, str]
```

new_row(fieldnames, d_invoice) creates a row with all columns initialized to blank and **dInvoice** initialized.

row_has_values(row, dimension_columns) prevents dimension-only empty rows from being appended.

The final output is:

```
rows: list[dict[str, str]]
```

This is written by **csv.DictWriter** using **fieldnames** from **BindingLayout**.

For a repeating child, **process_class(...)** completes and appends the parent row before processing **repeated_children**. It then calls **new_row(...)** for every child occurrence, including occurrence 1. Consequently the first child cannot share the parent row:

```

dAaa,dBbb,a1,a2,b1,b2,b3
1,,a1V1,a2V1,,,
1,1,,,b1V1,b2V1,b3V1
1,2,,,b1V2,b2V2,b3V2

```


When **dBbb** is non-repeating, **process_class(...)** passes the same **current_row** to the child and the result is instead:

```
1,,a1V1,a2V1,b1V1,b2V1,b3V1
```

Phase 1 Reverse Conversion: Structured CSV To UBL XML

Script:

```
src/syntax_binding.py --reverse
```

Main function:

```
write_xml_from_hierarchical_csv(...)
```

Input:

- Structured CSV.
- Same UBL syntax binding table.

Output:

- UBL Invoice XML.

Reverse Data Interpretation

Reverse mode reads:

```
rows: list[dict[str, str]]  
bindings: list[Binding]
```

Each Structured CSV row is sparse. Dimension columns determine where the row belongs in the XML hierarchy.

Example:

```
dInvoice=1  
dInvoiceLine=2  
InvoiceLineNetAmount=1000
```

This means the value belongs under the second **cac:InvoiceLine**.

Before XML construction, **validate_hierarchical_row_scopes(rows, fieldnames, field_dimension)** applies the ownership metadata built by **build_layout_from_rows(...)**:

1. It identifies dimension columns with **is_dimension_column(...)**.
2. For each input row, it takes the deepest populated dimension as the row scope.
3. For each populated fact, it looks up the owner in **BindingLayout.field_dimension**.
4. It raises **ValueError** when the owner differs from the row scope.

This rejects a mixed row such as **1,1,a1V1,a2V1,b1V1,b2V1,b3V1**. The parent facts must remain on the parent row and the repeating child facts must start on a separate child row.

XML Construction

The reverse writer creates:

```
root: ET.Element
```

It then writes values using binding XPath.

Important helper functions:

- **ensure_path(root, xpath, namespaces, force_new_leaf=False)**: walks an XPath and creates missing XML elements.
- **find_or_create_child(parent, step, namespaces, force_new=False)**: finds a child element matching a tag and predicate or creates it.
- **create_context(...)**: creates or reuses the XML parent context for a repeated row.
- **set_xml_value_with_currency(...)**: writes a value and applies **currencyID** when the target is an amount.
- **set_relative_xml_value(...)**: writes paths contained by the current repeated syntax context.
- An absolute XPath that remains outside the repeated context is passed to **set_xml_value_with_currency(...)** with the document root. This covers cross-scope mappings such as BT-90: its semantic path is below payment instructions, while its UBL XPath is below **AccountingSupplierParty**.
- **set_relative_xml_value(...)**: writes a value below a repeated context.
- **ensure_tax_scheme_defaults(...)**: adds required UBL tax scheme defaults.
- **load_ubl_child_order(...)**: reads UBL XSD files from **--ubl-schema-root** or **--ubl-schema-url** and builds child element order from **xs:sequence**.
- **sort_children_for_ubl_schema(...)**: reorders children using generated UBL schema order. If no schema source is supplied, it uses the built-in fallback order for the current PoC structures.

The reverse process is path-construction based. It uses the same binding table, but currently does not use the same recursive **BindingClass** tree as forward mode. Repeated-context paths are made relative only when the binding XPath is contained by the repeated syntax path; out-of-context

absolute paths remain document-rooted. It remains schema-order aware and can use UBL XSD order for UBL 2.1 through later UBL versions when a schema root or schema URL is supplied.

Phase 2 Syntax Binding: Structured CSV To ADS XBRL GL

Script:

```
src/syntax_binding_ads_xbri_gl.py
```

Main functions:

```
load_bindings(...)
validate_hierarchical_row_scopes(...)
build_instance(...)
convert_file(...)
```

Input:

- Phase 1 Structured CSV.
- ADS XBRL GL syntax binding CSV.
- Currency minor unit table, **specs/Currency.csv**.

Output:

- One of six ADS XBRL GL tuple views: **Invoices_Received.xbri**, **Invoices_Generated.xbri**, **Invoices_Received_Lines.xbri**, **Invoices_Generated_Lines.xbri**, **Supplier_Listing.xbri**, or **Customer_Master.xbri**.

Binding Interpretation

ADS XBRL GL bindings use **semantic_path** and **structured_csv_column** to select the source Structured CSV value. They use **element** and **xpath** to identify the target XBRL GL item and tuple location.

Before building the XBRL GL instance, **validate_hierarchical_row_scopes(...)** derives each fact's owning dimension from binding **type=C**, **multiplicity**, and **semantic_path** rows. A non-repeating child belongs to its nearest repeating ancestor row. A repeating child owns a separate row from its first occurrence. The converter rejects input that populates parent facts and repeating-child facts on the same child row.

Example binding dictionary:

```
{
  "source_column": "InvoiceLineIdentifier",
  "semantic_path": "$.invoice.invoiceLine.invoiceLineIdentifier",
```

```

    "element": "gl-cor:documentNumber",
    "xpath": "/xbrli:xbrl/gl-cor:accountingEntries/...",
    "value_source": "structured_csv_column",
    "unit_ref_rule": "",
    "decimals": ""
}

```

Unlike **syntax_binding.py**, this script keeps binding rows as dictionaries.

```

bindings: list[dict[str, str]]
rows: list[dict[str, str]]
currency_minor_units: dict[str, str]

```

ADS Dimension Ownership And Input Validation

load_bindings(binding_csv) retains the binding properties needed for both XBRL construction and Structured CSV validation:

- **type** distinguishes class (**C**) and fact (**A**) rows;
- **multiplicity** determines whether a class repeats;
- **semantic_path** defines class ancestry;
- **structured_csv_column** becomes **source_column**;
- **element** and **xpath** identify the XBRL GL target.

validate_hierarchical_row_scopes(rows, bindings) performs these steps:

1. It scans **type=C** rows. The invoice root and classes for which **multiplicity_repeats(...)** is true are registered in **class_dimensions**.
2. **dimension_name(source_column)** converts a class column such as **InvoiceLine** into **dInvoiceLine**.
3. For every **type=A** fact, it removes semantic path segments from the right until it finds the nearest registered class dimension. The result is stored in **field_dimensions**.
4. For each Structured CSV row, it finds the deepest populated **dXxx** column.
5. It rejects the row if any populated ADS-bound fact belongs to a different dimension.

This means a single child class has no independent row scope and its facts belong to the nearest repeating ancestor. A repeating child has its own row scope from occurrence 1. Parent facts and repeating-child facts cannot be populated together on that first child row.

The validation is called by **convert_file(...)** immediately after **read_csv(...)** and before **build_instance(...)**, so invalid hierarchy is not partially written to an XBRL GL instance.

ADS Row Selection

After validation, **rows_for_binding(rows, binding)** selects the sparse input rows used by each fact binding:

- VAT breakdown facts require a populated **dVatBreakdown**;
- invoice-line facts require a populated **dInvoiceLine**;
- document-level facts use the first row containing the source value.

first_non_empty(...) obtains document-wide currency, date, and entity values. The selected source row is also passed to **unit_ref_from_rule(...)**, so a monetary fact can obtain its unit from a bound Structured CSV currency column.

XBRL XML Construction

build_instance(...) creates an **ET.Element** tree rooted at **xbrli:xbrl**. It adds context, units, XBRL GL document metadata, and the tuple containers required by the selected binding paths. Invoice bindings create document structures; Supplier Listing and Customer Master create the applicable identifier reference structures.

Path handling:

- **path_parts(xpath)** splits the target XPath into steps.
- **parse_path_step(step)** separates an element name from selector conditions.
- **container_for_path(root, xpath, currency, monetary_decimals)** walks the tuple path.
- **ensure_child(...)** finds or creates tuple containers.
- **selector_matches(...)** checks whether an existing element satisfies selector child values.
- **append_item(...)** creates the final item element and writes the value.

The Supplier Listing binding selects the vendor identifier with **gl-cor:identifierReference[gl-cor:identifierType="V"]**. Seller postal address facts use target paths below that selected identifier reference and create **gl-bus:identifierAddress** with **identifierStreet**, **identifierAddressStreet2**, **identifierCity**, **identifierStateOrProvince**, **identifierCountry**, and **identifierZipOrPostalCode** children. The selector identifies the vendor tuple; it does not create address children unless corresponding fact rows are present in the binding table.

The Seller address must not be written as **gl-bus:organizationAddress**. That tuple describes the reporting entity under **entityInformation**. A supplier represented by the vendor identifier reference uses **gl-bus:identifierAddress** below the same **identifierType=V** tuple.

Rows are selected by **rows_for_binding(rows, binding)**. A binding for invoice lines selects rows where the relevant source column and line dimension are present. A document-level binding selects the document row.

Numeric facts use:

- **unit_ref_from_rule(...)** for **unitRef**.
- **decimals_for_item(...)** for **decimals**.
- **load_currency_minor_units(...)** to get currency minor units from **specs/Currency.csv**.
- **add_missing_units(...)** to ensure every referenced unit is declared.

Construction then finishes as follows:

1. **make_xbrl_root(...)** creates **xbrli:xbrl** and **schemaRef**.

2. **add_context_and_units(...)** adds the default context and initial units.
3. **add_document_info(...)** writes required XBRL GL document metadata.
4. **build_instance(...)** writes VAT occurrences and other bound facts.
5. **reorder_tree(...)** applies the supported XBRL GL child order, including **identifierAddress** under **identifierReference** and the schema order of the address facts.
6. **add_missing_units(...)** declares any unit referenced by generated facts.
7. **convert_file(...)** indents and writes the XML document.
8. **main(...)** resolves one file or directory with **input_files(...)** and derives output names with **binding_target_stem(...)** and **schema_href_for_output(...)**.

Phase 2 Semantic Binding: Structured CSV To PSV Or CSV

Script:

```
src/semantic_binding.py
```

Main functions:

```
load_bindings(...)
transform_rows(...)
transform_repeated_group_rows(...)
merge_values(...)
convert_file(...)
```

Input:

- Phase 1 Structured CSV.
- Semantic binding CSV under **specs/bindings/semantic/**.

Output:

- ADS PSV or CSV target file.

Semantic Binding Table

The semantic binding table starts from the target definition columns and adds UADC mapping columns.

```
field_no,field_name,level,flat_file_data_type,length,description,source_document,semantic_path,type,multiplicity
```

Target field rows use **type=A**. Class rows use **type=C**. The class rows are not output columns. They tell the script which semantic classes repeat.

Runtime Objects

read_csv_rows(path) returns:

```
list[dict[str, str]]
```

load_bindings(binding_csv) builds:

```
repeated_classes: dict[str, SemanticClass]  
bindings: list[SemanticBinding]
```

SemanticClass holds:

- **semantic_path**;
- **multiplicity**.

Only repeated class rows are kept in **repeated_classes**.

SemanticBinding holds:

- **target_column**;
- **semantic_path**;
- **normalized_semantic_path**;
- **source_column**;
- **repeat_group_path**;
- **repeat_group_column**;
- **repeat_index**.

Source Column Derivation

The source Structured CSV column is derived from the final segment of **semantic_path**.

```
$.invoice.invoiceNumber -> InvoiceNumber  
$.invoice.invoiceLine.invoiceLineIdentifier -> InvoiceLineIdentifier
```

Indexed paths are normalized first.

```
$.invoice.vatBreakdown[0].vatCategoryTaxAmount
-> normalized path: $.invoice.vatBreakdown.vatCategoryTaxAmount
-> source column: VatCategoryTaxAmount
-> repeat group path: $.invoice.vatBreakdown
-> repeat group column: dVatBreakdown
-> repeat index: 0
```

Semantic Dimension Ownership Resolution

Semantic binding resolves fact ownership from the same three binding properties used by syntax binding: **type=C**, **multiplicity**, and **semantic_path**.

1. **load_bindings(...)** reads all rows with **read_csv_rows(...)**.
2. For each **type=C** row, **multiplicity_repeats(...)** decides whether the class defines a repeated row scope. Only repeating classes are placed in **repeated_classes** because a single class does not create a Structured CSV dimension row.
3. **indexed_semantic_path(...)** removes an explicit occurrence selector such as **[0]** while preserving its zero-based **repeat_index**.
4. **source_column_for_path(...)** maps the last semantic path segment to the Structured CSV fact column.
5. **deepest_repeated_class_ancestor(...)** finds the nearest repeated class above each fact.
6. **dimension_column_for_path(...)** converts that class path to its **dXxx** column.
7. **resolve_bindings_for_classes(...)** stores the result in **SemanticBinding.repeat_group_path** and **SemanticBinding.repeat_group_column**.

Thus a fact below a non-repeating child is assigned to the nearest repeating ancestor. A fact below a repeating child is assigned to that child's own dimension from its first occurrence. This ownership controls which sparse Structured CSV rows may supply the fact; it does not by itself copy parent facts onto a Structured CSV child row.

Row Scope Selection

resolve_bindings_for_classes(bindings, repeated_classes) assigns row-scope information to each **SemanticBinding**. For non-indexed paths, it searches the deepest repeated **C** ancestor in the binding table.

Example:

```
semantic_path: $.invoice.invoiceLine.invoiceLineNetAmount
repeated class: $.invoice.invoiceLine
repeat_group_column: dInvoiceLine
```

row_scope_group(bindings) determines whether the target file emits one row per repeated occurrence.

Rules:

1. Indexed bindings are ignored as row-scope candidates.
2. Non-indexed bindings with repeated class ancestors are candidates.
3. The deepest repeated class wins.

Thus **Invoices_Received_Lines** emits one row per **dInvoiceLine**, while **Invoices_Received** stays at invoice level even though it may contain indexed VAT breakdown columns.

Transformation Without Repeated Row Scope

If **row_scope_group(...)** returns no scope, **transform_rows(...)** creates invoice-level target rows.

Working variables:

- **output_rows:** list[dict[str, str]];
- **current_row:** dict[str, str] | None;
- **repeat_counts:** dict[str, int];
- **key_binding:** SemanticBinding.

The script scans Structured CSV rows. **row_has_bound_data(...)** skips irrelevant sparse rows. **row_repeat_indices(...)** assigns occurrence numbers for indexed groups. **merge_values(...)** copies non-empty values into the current target row.

Transformation With Repeated Row Scope

If **row_scope_group(...)** returns a scope, **transform_repeated_group_rows(...)** emits one target row per repeated occurrence.

Example:

```
scope_path=$.invoice.invoiceLine
scope_column=dInvoiceLine
```

Working variables:

- **parent_context:** dict[str, str]: invoice-level values.
- **current_group_row:** dict[str, str] | None: current repeated output row.
- **current_scope_value:** str: current **dXxx** occurrence value.
- **parent_bindings:** list[SemanticBinding].
- **scoped_bindings:** list[SemanticBinding].

Rows with **dInvoice** and no selected **scope_column** fill **parent_context**. Rows with the selected **scope_column** start or continue a repeated output row. When the **scope_column** value changes, the current target row is appended to **output_rows** and a new row starts from **parent_context**.

The first populated repeated dimension value therefore starts the first independent target record. Parent values are copied from **parent_context** into that flat target record; they are not expected to

be duplicated on the input Structured CSV child row. This distinction preserves the hierarchical input rule while still producing self-contained ADS line records.

Value Merge Rules

merge_values(target_row, source_row, bindings, repeat_indices) is the only function that writes values into a target row.

Rules:

1. Skip bindings without **source_column**.
2. If **repeat_index** exists, require the current occurrence number to match.
3. Read **source_row[source_column]**.
4. Use **default_value** if the source is blank.
5. Write to **target_row[target_column]** only if the target cell is still blank.

This first-non-empty rule is important because Structured CSV rows are sparse and hierarchical.

Semantic Binding Output And Orchestration

1. **row_has_bound_data(...)** ignores sparse rows irrelevant to the selected target view.
2. **row_scope_group(...)** selects invoice-level transformation or the deepest non-indexed repeated target scope.
3. **row_repeat_indices(...)** and **repeated_group_present(...)** maintain zero-based indices used by explicitly indexed semantic bindings.
4. **transform_rows(...)** dispatches to invoice-level merging or **transform_repeated_group_rows(...)**.
5. **write_target_file(...)** writes binding-order columns with the selected delimiter.
6. **output_format_options(...)** resolves PSV or CSV defaults and command-line overrides.
7. **output_path(...)** and **binding_target_stem(...)** derive the target file location and name.
8. **convert_file(...)** performs one input-file conversion, while **main(...)** expands directory inputs with **input_files(...)**.

Function-Level Map

Function	Script	Role
write_hierarchical_csv	syntax_binding.py	Converts UBL XML to Structured CSV and metadata.
build_layout_from_rows	syntax_binding.py	Derives dimensions and fact ownership from C/A , multiplicity, and semantic paths.
multiplicity_repeats	syntax_binding.py	Determines whether a class creates a repeated

Function	Script	Role
		dimension.
build_binding_class_tree	syntax_binding.py	Builds semantic class tree from C rows.
process_class	syntax_binding.py	Recursively walks XML contexts and creates rows.
fill_direct_fields	syntax_binding.py	Extracts direct A values for the current class.
validate_hierarchical_row_scopes	syntax_binding.py	Rejects mixed parent/repeating-child rows before reverse conversion.
write_xml_from_hierarchical_csv	syntax_binding.py	Converts Structured CSV back to UBL XML.
load_bindings	syntax_binding_ads_xbrl_gl.py	Loads ADS class/fact properties and XBRL target paths.
validate_hierarchical_row_scopes	syntax_binding_ads_xbrl_gl.py	Derives ADS fact ownership and rejects mixed input rows.
rows_for_binding	syntax_binding_ads_xbrl_gl.py	Selects document, VAT, or invoice-line source rows.
build_instance	syntax_binding_ads_xbrl_gl.py	Builds one ADS XBRL GL tuple instance.
container_for_path	syntax_binding_ads_xbrl_gl.py	Finds or creates XBRL GL tuple containers from XPath.
append_item	syntax_binding_ads_xbrl_gl.py	Writes one XBRL GL item.
load_bindings	semantic_binding.py	Loads semantic binding rows and resolves source columns and row scopes.

Function	Script	Role
deepest_repeated_class_ancestor	semantic_binding.py	Finds the nearest repeated class owning a fact.
resolve_bindings_for_classes	semantic_binding.py	Assigns source columns and repeated dimensions to target bindings.
row_scope_group	semantic_binding.py	Selects the repeated class used for target record emission.
transform_rows	semantic_binding.py	Creates target rows from Structured CSV.
transform_repeated_group_rows	semantic_binding.py	Emits one target row per repeated dimension.
merge_values	semantic_binding.py	Copies source values into target cells.
convert_file	all three scripts	Runs one input-file conversion and writes the script-specific output.

Source Documents

Detailed binding-table entry rules are documented in **docs/syntax_binding_conversion/README.md** and **docs/semantic_binding_conversion/README.md**.

Operational command examples are documented in **docs/development/README.md** and **docs/testing/README.md**.